Documentación Técnica Breve

Mi Contador de Bolsillo: arquitectura confiable para preguntas de negocio

El sistema permite que un microcomerciante pregunte en lenguaje natural sobre ventas, clientes, productos y tendencias. El backend calcula las métricas con reglas determinísticas sobre el dataset y el LLM solo ayuda a clasificar preguntas ambiguas y explicar los resultados en español claro.

La solución está diseñada para personas sin educación financiera formal: responde con números exactos, visualizaciones simples y recomendaciones accionables, sin obligar al usuario a leer dashboards complejos.

Angular 20	Express TS	19 intents	100%	7/7
frontend chat, voz y gráficos	API REST y orquestación	ventas, clientes, productos y ayuda	15 pruebas principales	anexos de barreras

Proyecto: Mi Contador de Bolsillo - DeUna
Equipo: 4NOVA
Entregable: Documento corto con arquitectura, modelo/LLM, prompts, conexión al dataset y manejo de errores.
Fecha: 19 de abril de 2026
Estado: Prototipo funcional con interfaz chat, voz, modo Simple/Pro, analítica determinística, fechas relativas dinámicas y reformulación con IA.

1. Resumen Ejecutivo

La arquitectura separa con claridad las responsabilidades:

Interfaz Angular
Chat, entrada por voz, selector visual, modo Simple/Pro y render de visualizaciones.

→

API Express
Recibe la pregunta, valida payload, selecciona comercio y coordina intent, cálculo y LLM.

→

Motor determinístico
Calcula ventas, tendencias, clientes, ticket, categorías y alertas desde JSON.

→

LLM controlado
Reformula resultados ya calculados. No recibe el dataset completo ni inventa métricas.

Actualización importante: el proyecto ahora usa una utilidad centralizada de fecha real de Ecuador. Cuando el usuario dice "hoy", "ayer", "esta semana" o "este mes", la app calcula el periodo con `America/Guayaquil`; no usa una fecha quemada ni la última fecha del dataset.

2. Stack Tecnológico

Capa	Tecnología	Uso dentro del prototipo
Frontend	Angular 20 standalone	Chat tipo app móvil DeUna, modo Simple/Pro, voz y visualizaciones.
Backend	Node.js + Express + TypeScript	API REST, intents, analítica, manejo de errores y conexión LLM.
Datos	JSON en memoria	Dataset sintético de transacciones en `server/src/data/transactions.json`.
IA / LLM	OpenAI GPT-4o-mini configurable	Reformulación natural, clasificación fallback y manejo de consultas generales.
Fallback sin IA	NoOpProvider	Respuestas directas y controladas si no hay API key o se desactiva OpenAI.
Caché	LLMCache en memoria	Evita llamadas repetidas al LLM por intent y reformulación.
Voz	Web Speech API	Reduce barreras de escritura para microcomerciantes.
Visualizaciones	HTML/CSS en Angular	Barras, líneas, tablas y pastel/progreso sin librerías pesadas.

3. Arquitectura General

Usuario
Pregunta en lenguaje natural: "¿Cuánto vendí esta semana?"

→

Frontend
Envía `{ message, mode, merchantId }` a la API.

→

Chat Route
Valida, detecta intent, ejecuta motor y arma respuesta.

→

Respuesta
Devuelve answer, intent, métricas, follow-ups y visualización.

Dataset JSON
3 comercios y 15,042 transacciones totales; demo fuerza `m001`.

→

Data Loader
Normaliza formato, filtra completadas y entrega Tienda Don Pepe.

→

Analytics Engine
Produce `{ value, label, metricsUsed, visualization }`.

→

LLM Adapter
Solo reformula el label calculado o clasifica si las reglas no bastan.

4. Mapa de Módulos del Backend

index.ts
Inicializa Express, CORS, JSON parser, dataset y proveedor LLM.

chat.routes.ts
Endpoint principal. Orquesta intent, cálculo, LLM, follow-ups y visualización.

analytics.engine.ts
Funciones determinísticas para ventas, clientes, tendencia, ticket y productos.

data-loader.ts
Carga JSON desde `src` o desde build `dist`; fuerza m001 para demo.

intent-catalog.ts
Catálogo de 19 intents con keywords, regex, descripción y follow-ups.

intent-router.ts
Detecta intent y extrae fechas/años con tolerancia a errores.

date-utils.ts
Fecha real de Ecuador, formato local y etiquetas de fecha.

llm/*
OpenAIProvider, NoopProvider, LLMCache y adaptador común.

5. Flujo de una Consulta

1. Input
El usuario pregunta: "¿Cuánto vendí ayer?"

2. Intent
Regex/keywords detectan `sales_specific_date`.

3. Fecha
`extractDateFromMessage` convierte "ayer" a fecha real -1 día.

4. Métrica
`getSalesForDate` filtra transacciones completadas.

5. Lenguaje
LLM reformula el label calculado según Simple/Pro.

6. UI
Angular muestra respuesta, follow-ups y gráfico si aplica.

El endpoint principal es `POST /api/chat`. Su respuesta incluye:

Campo	Descripción
`answer`	Respuesta final que ve el usuario.
`mode`	`simple` o `complete`.
`detectedIntent`	Intent clasificado por reglas o LLM.
`confidence`	Confianza de clasificación.
`metricsUsed`	Métricas determinísticas usadas.
`suggestedFollowUps`	Preguntas sugeridas.
`visualization`	`bar`, `line`, `pie`, `table` o `null`.

6. Fechas Relativas y Zona Horaria

Antes, la lógica podía confundirse si "hoy" se trataba como una fecha fija o como la última fecha del dataset. Ahora está centralizada:

getCurrentEcuadorDate()
Obtiene fecha/hora real en `America/Guayaquil`.

→

chat.routes.ts
Pasa la referencia real al parser de fechas.

→

intent-router.ts
Resuelve "hoy", "ayer", "aller" y "anteayer".

→

analytics.engine.ts
Calcula día, semana, mes, tendencia y churn con esa referencia.

Expresión	Manejo actual
`hoy`, `día de hoy`, `día actual`	Se convierte a la fecha real de Ecuador del momento de consulta.
`ayer`, `aller`	Se interpreta como fecha real de Ecuador menos un día.
`anteayer`, `ante ayer`	Se interpreta como fecha real de Ecuador menos dos días.
`esta semana`	Rango lunes-domingo de la semana actual.
`este mes`	Mes calendario de la fecha real.
Fechas explícitas	Se respetan, por ejemplo `2025`, `17 de abril`, `2025-04-17`.

> Esto quedó validado en el set de pruebas: "¿Cuánto he vendido el día de hoy?", "¿Cuánto vendí ayer?", "caunto bendi aller" y "¿Cuánto vendí esta semana?" pasan contra el endpoint real.

7. Dataset y Conexión a Datos

El archivo `server/src/data/transactions.json` contiene 3 comercios. El prototipo fuerza `m001` para que la demo sea estable y todas las pruebas correspondan a Tienda Don Pepe.

Comercio	Categoría	Transacciones totales	Completadas	Ventas completadas	Rango
Tienda Don Pepe (`m001`)	abarrotes	2,962	2,827	$118,048.67	2024-01-01 a 2026-04-19
Café Internet Mary (`m002`)	servicios	4,777	4,548	$56,277.58	2024-01-01 a 2026-04-19
Restaurante Doña Lupita (`m003`)	restaurante	7,303	6,921	$157,519.90	2024-01-01 a 2026-04-19

Carga actual	`data-loader.ts` lee JSON en memoria y normaliza si viene como array, objeto con `merchants` o un solo comercio.
Compatibilidad build	Ahora busca `transactions.json` junto al build y también en `src/data`, evitando fallos al correr desde `dist`.
Demo controlada	`getMerchantData` y `getAllMerchants` filtran a `m001`; el selector visual no implica multi-comercio activo en backend.
Migración futura	Se puede reemplazar solo `data-loader.ts` por APIs reales de DeUna manteniendo intents y motor analítico.
Privacidad	El dataset completo no viaja al LLM. Solo se envían labels calculados y métricas usadas.

8. Motor Analítico Determinístico

El motor en `analytics.engine.ts` evita que el LLM haga aritmética. Cada función filtra transacciones completadas y devuelve un objeto uniforme.

Función	Qué calcula	Visualización
`getSalesToday` / `getSalesForDate`	Total y conteo de ventas por día.	Barra por hora
`getSalesForPeriod`	Ventas de semana o mes actual.	Barra por día
`getSalesForYear`	Total anual y transacciones por año.	Línea mensual
`comparePeriods`	Mes actual vs anterior, diferencia y variación.	Barra comparativa
`getBestDay` / `getWorstDay`	Días extremos del historial.	Sin gráfica
`getAverageTicket`	Promedio por transacción.	Sin gráfica
`getChurnRisk`	Clientes frecuentes sin compras recientes.	Tabla si hay riesgo
`getRepeatCustomers`	Clientes por frecuencia y total gastado.	Tabla
`getDayOfWeekAnalysis`	Promedio por día de semana.	Barra
`getSalesTrend` / `detectSignificantChange`	Últimos 14 días y cambio porcentual.	Línea
`getTopProducts`	Categorías líderes por ventas.	Pastel/progreso
`getPaymentBreakdown`	Total por método de pago.	Pastel/progreso

9. Intents Soportados

El catálogo actual contiene 19 intents. Se cubre más que el mínimo del reto y se separan preguntas de negocio de barreras conversacionales.

Grupo	Intents
Ventas por tiempo	`sales_today` , `sales_specific_date` , `sales_specific_year` , `sales_this_week` , `sales_this_month`
Comparación y tendencia	`sales_comparison` , `sales_trend` , `significant_change` , `best_day` , `worst_day` , `strong_weak_days`
Clientes	`customer_churn` , `repeat_customers`
Producto y pago	`top_products` , `payment_methods` , `average_ticket`
Asistencia	`proactive_alert` , `greeting` , `help`

> El intent `sales_today` se reforzó para frases como "ventas del día de hoy", "cuánto he vendido en el día de hoy" y "hoy cuánto he vendido". Esto evita depender de una sola frase exacta.

10. LLM, Prompts y Barreras

El proveedor principal es `OpenAIProvider` , con modelo configurable por `LLM_MODEL` y valor por defecto `gpt-4o-mini` . El backend usa OpenAI solo si `LLM_PROVIDER=openai` y existe `OPENAI_API_KEY` ; si no, usa `NoopProvider` .

Uso del LLM	Prompt / regla	Valor técnico
Reformulación	No inventar datos, hablar en segunda persona, tono amigable, Simple breve y Pro con contexto.	Convierte métricas en lenguaje claro para microcomerciantes.
Clasificación fallback	Responder solo el nombre del intent, tolerar errores ortográficos, distinguir fechas y periodos.	Ayuda en inputs informales o ambiguos.
Consulta general	Acepta saludos y cálculos de caja; rechaza temas fuera del negocio.	Mantiene foco y evita que el producto se vuelva asistente general.
Fallback sin IA	`NoopProvider` responde con labels directos y sugerencias.	La demo no se rompe si no hay API key.

Por qué se rechazan preguntas fuera de dominio

Aunque un LLM conectado a OpenAI pueda saber quién juega o quién ganó un partido, ese no es el objetivo del chatbot. La barrera de dominio protege la experiencia: el asistente debe concentrarse en ventas, clientes, caja, productos y salud del negocio.

11. Modos Simple y Pro

Modo	Comportamiento	Caso de uso
Simple (`simple`)	Respuesta breve de 1-2 frases, con baja carga cognitiva.	Comerciante atendiendo, necesita el dato rápido.
Pro (`complete`)	Más contexto, recomendación y visualización si aplica.	Demo, análisis, revisión de tendencia y toma de decisiones.

La interfaz envía el modo al backend. El LLM lo usa para ajustar extensión y profundidad, pero los números siguen saliendo del motor determinístico.

12. Visualizaciones

Las visualizaciones son datos estructurados generados por backend y renderizados en Angular. No son obligatorias para todo: aparecen cuando aclaran el resultado.

Tipo	Uso	Ejemplo de pregunta
`bar`	Ventas por hora/día, semana, mes o días fuertes.	"¿Cuánto vendí esta semana?"
`line`	Tendencia reciente o ventas anuales.	"¿Cuál es la tendencia?"
`table`	Clientes frecuentes o en riesgo.	"¿Quiénes me compran más?"
`pie`	Categorías o métodos de pago.	"¿Qué categorías se venden más?"

> Para evitar cortes feos en este PDF, las cajas, diagramas, tablas y tarjetas usan `break-inside: avoid` y `page-break-inside: avoid`. Además, las secciones largas se separan con saltos controlados.

13. Manejo de Errores y Casos Borde

Caso	Manejo implementado
Mensaje vacío	`POST /api/chat` responde 400 con "El mensaje no puede estar vacío".
Intent desconocido	Si hay LLM, responde dentro del dominio; si no, sugiere preguntas válidas.
Pregunta fuera de dominio	Rechazo cordial y foco en ventas/salud del negocio.
Fecha relativa	`hoy`, `ayer`, `aller`, `anteayer` se resuelven con fecha real de Ecuador.
Fecha no parseada	En `sales_specific_date`, fallback a ventas del mes.
Fecha sin ventas	Responde que no hay ventas y no inventa gráfico.
Clientes sin churn	Responde que no hay clientes en riesgo.
Error OpenAI	Captura error y devuelve label determinístico original.
Sin API key	Usa `NoopProvider` con respuestas directas.
Dataset no encontrado en build	`data-loader.ts` prueba rutas candidatas antes de fallar.
Error interno	HTTP 500 con "Error procesando tu pregunta".

14. Alertas Proactivas

El módulo `server/src/alerts/proactive-alerts.ts` genera insights sin que el usuario pregunte.

Alerta	Regla
Caída de ventas	Variación reciente menor a -15%.
Ventas en subida	Variación reciente mayor a 20%.
Dependencia de cliente	Un cliente concentra más de 30% de ventas.
Clientes en riesgo	Cliente frecuente sin compra en más de 30 días.
Día fuerte/flojo	Sugiere promociones en el día con menor promedio.

En la corrida de Tienda Don Pepe, el consejo proactivo identifica subida reciente de 20.4%, martes como día fuerte y domingo como día flojo.

15. Endpoints

Método	Ruta	Descripción
`POST`	`/api/chat`	Procesa una pregunta y devuelve respuesta, intent, métricas, follow-ups y visualización.
`GET`	`/api/proactive-alert`	Devuelve la alerta proactiva principal.
`GET`	`/api/sample-questions`	Lista preguntas sugeridas.
`GET`	`/api/merchants`	Devuelve comercios disponibles; en demo solo `m001` .
`GET`	`/api/health`	Verifica estado del backend.

16. Pruebas y Validación

La validación se ejecuta con `node run_tests.js` . El runner ahora calcula expectativas dinámicas desde el dataset y la fecha real de Ecuador. Los resultados quedan en `test_results.json` y alimentan el PDF de pruebas.

Grupo	Resultado de la última corrida
15 preguntas principales de negocio	15 / 15 correctas
Anexos de barreras y modos	7 / 7 correctas
Acierto principal	100%
Acierto anexos	100%
Requisito mínimo del reto	80%

Relación con el PDF de pruebas

`Set_Pruebas_15_Preguntas_4NOVA_DeUna.pdf` documenta input, modo, esperado, output real, relevancia, observaciones y código de evidencia. Esta documentación técnica explica cómo la arquitectura produce esos resultados.

17. Seguridad, Costos y Desempeño

Precisión	Los cálculos no dependen del LLM. El backend calcula y el LLM reformula.
Privacidad	El dataset completo permanece en servidor; solo se envían resultados resumidos al LLM.
Costo	`LLMCache` evita llamadas repetidas para intents y reformulaciones ya vistas.
Latencia	Regex/keywords y JSON en memoria son rápidos; el LLM es una capa de lenguaje.
Escalabilidad	Puede migrar a API real de DeUna reemplazando `data-loader.ts`.
Operación sin LLM	`NoopProvider` mantiene el prototipo utilizable con respuestas directas.

18. Limitaciones y Próximos Pasos

Limitación actual	Siguiente paso recomendado
Backend fuerza `m001`.	Habilitar selección real por `merchantId` cuando se conecte a datos reales.
Dataset local estático.	Reemplazar JSON por API transaccional DeUna manteniendo contrato `Transaction`.
Caché en memoria.	Migrar a Redis si se despliega en múltiples instancias.
Métricas dependen de campos disponibles.	Agregar margen, costo, inventario y canal cuando existan en fuente real.
Visualizaciones simples.	Mantener simplicidad para microcomercio o migrar a librería ligera si se requiere interacción avanzada.
Sin autenticación productiva.	Agregar auth, control de comercio por usuario y permisos antes de producción.

19. Conclusión Técnica

La solución cumple el reto porque combina datos exactos, lenguaje accesible y visualizaciones simples. La arquitectura evita que el LLM invente números: el backend calcula, el LLM explica y la interfaz muestra. La actualización de fechas dinámicas deja el prototipo listo para operar con expresiones naturales como "hoy" y "ayer" sin depender de una fecha fija.